

Puzzle 135 — The 21-Input Sweep Transaction and the Shared-z Multi-Key Lattice Analysis

Standalone report derived from the new section at the bottom of LOG2.txt (added 2026-08-01), with independent empirical verification against the real on-chain transaction.

1. The New Content in LOG2.txt (transcribed)

1.1 The claim

If multiple (r, s, z) signatures are bundled within the same Bitcoin transaction, they represent multiple inputs being spent simultaneously. The message hash z is claimed to be identical (or highly structurally linked) across them, because Bitcoin signatures commit to the entire transaction template via SIGHASH_ALL. Multiple signatures under an identical z for different private keys would transform the puzzle from an underdetermined single-signature search into a constrained simultaneous lattice system.

1.2 The multi-key linear intersection

Two inputs signed by two different keys $d1, d2$:

$$s1*k1 - r1*d1 = z \pmod{N}$$

$$s2*k2 - r2*d2 = z \pmod{N}$$

Identical z eliminates it by subtraction:

$$s1*k1 - s2*k2 = r1*d1 - r2*d2 \pmod{N}$$

With joint carry multiplier m_joint :

$$s1*k1 - s2*k2 - r1*d1 + r2*d2 = m_joint * N$$

1.3 The bounding claim

With d_i in $[2^{134}, 2^{135})$ and k_i in $[2^{250}, 2^{256})$, the log claims m_joint "collapses entirely into a tiny, narrow envelope."

1.4 The $u = 2^{(1/10)}$ reaction

With $N = u^{2560} - \delta_N$:

$$\begin{aligned} & (s1*k1 - s2*k2)/u^{2560} - (r1*d1 - r2*d2)/u^{2560} \\ & = m_joint - \{ m_joint * \delta_N / 2^{256} \} \end{aligned}$$

Claim: $|d1 - d2| < 135$ bits and $|k1 - k2| < 6$ bits, so the combined fractional weight cannot clear one 25.6-bit tier, pinning the fractional part $\{-m_joint * \delta_N / 2^{256}\}$ to the radix residuals (19, -342.3).

1.5 The 5D joint lattice

```
B_Joint =
[ K_scale, 0, 0, 0, floor(s1*W) ]
[ 0, K_scale, 0, 0, floor(-s2*W) ]
[ 0, 0, D_scale, 0, floor(-r1*W) ]
[ 0, 0, 0, D_scale, floor(r2*W) ]
[ 0, 0, 0, 0, floor(-N*W) ]
```

LLL targets $x = [k1, k2, d1, d2, m_joint]$, claimed to unmask both nonces and both keys in one reduction.

2. Empirical Verification (this report)

2.1 The transaction is real and it matches the scenario

Puzzle 135's spend transaction:

```
17e4e323cfbc68d7f0071cad09364e8193eedf8fefbcb8a21b4b65717a4b3d3
```

On-chain facts (Blockstream API):

- 21 inputs, all spending Puzzle-series addresses (65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125, 130, 135, 140, 145, 150, 155, 160).
- P135 is input index 14 (address 16RGFo6hjq9ym6Pj7N5H7L1NR1rVPJyw2v).
- Input 20 (1PvaqLqRAIvje7CactLR55xQBYvBeaDrXN, 250,000 BTC) is not a catalog puzzle address.
- All 21 scripts have sighash byte 0x01 (SIGHASH_ALL).
- 10 of 20 puzzle keys are solved (65..130, keys known); 10 are unsolved (135..160).

So the empirical situation the log describes EXISTS: many puzzle keys bundled in one transaction. This is the correct transaction to test the theory on.

2.2 FINDING 1 — the identical-z premise is FALSE

The 20 cached z values are all distinct. For all 14 solved-with-RSZ entries, $k = (z + r*d)/s \bmod N$ passes the $kG.x == r$ check, proving each z is the true per-input sighash.

Why: legacy P2PKH sighash commits to the input's own scriptPubKey (script code). Distinct source addresses => distinct z, even under SIGHASH_ALL.

Identical z requires two inputs from the SAME address.

Therefore the subtraction trick (eliminate z) does NOT apply to this transaction. The 5D joint lattice as constructed cannot use it.

2.3 FINDING 2 — the "tiny envelope" claim is FALSE

Computed m_{joint} over the stated bounds with actual s1, s2, r1, r2:

```
m_joint min ~ -1.39e76
m_joint max ~  1.53e76
width      ~  2.92e76  (10^76.47)
```

Same enormous $\sim 10^{76}$ envelope as the single-signature case. The "6-bit nonce headroom" still spans 2^{250} absolute values; scaled by s ($\sim 1.5e76$) it dominates. The "cannot clear a single 25.6-bit tier" claim is not supported by the numbers.

2.4 FINDING 3 — RFC6979 nonce pattern (the real discovery)

- RFC6979 implementation validated against the official SEC test vector ($x = 0xC9AFA9D8\dots$, $h = \text{SHA256}(\text{"sample"})$, expected $k = 0xA6E3C57D\dots$; PASS).
- Exactly 4 of 14 solved keys have nonces that match RFC6979(d_i, z_i):

```
Puzzle 85, 90, 120, 125 -> k = RFC6979(d, own z) EXACT
```
- The other 10 (65, 70, 75, 80, 95, 100, 105, 110, 115, 130) do NOT match.
- Hypothesis B (all nonces = RFC6979(d_i, Z_{shared})): REJECTED — only self-hits.
- Hypothesis D (RFC6979 with extra entropy $0x00..0x04$): REJECTED.

Interpretation: the 21-input transaction was signed by a MIX of at least two signing schemes. Puzzles 85, 90, 120, 125 were signed by a standard deterministic RFC6979 signer; the remaining inputs used something else (random nonces, or a non-standard scheme).

2.5 Why this matters for Puzzle 135

The same signing session produced:

- 10 verified (d, k, z, r, s) tuples (known keys),
- 10 unknown-key signatures (135..160), including P135 at input 14.

If the unknown-key inputs were signed by the SAME tool that produced the RFC6979 matches, their nonces are deterministic functions of (d_i, z_i) and

are recoverable once a candidate d_i is proposed. The batch structure (every 5th puzzle, 65..160) and the mixed-signer pattern give a concrete testable hypothesis set:

```
H1: k_135 = RFC6979(d_135, z_135)          (d_135 unknown, z_135 known)
H2: k_135 drawn from same RNG as 65..130   (non-RFC6979 signer)
H3: signer alternates or batches -&gt; k_135 scheme inferred from neighbors
    (input 13 = P130 [solved, non-RFC6979], input 15 = P140 [unsolved])
```

Each hypothesis yields a candidate k_{135} (or a distribution), which converts immediately to a candidate private key:

$$d_{135} = (k_{135} * s - z) / r \pmod{N}$$

and is verifiable by checking $d_{135} * G == P_{135}$.

3. Conclusion

The new LOG2.txt section correctly identified a real structural feature of the Puzzle series (multi-key bundled spending transactions), but its two core mathematical premises are falsified empirically:

1. z is NOT shared across inputs of the P135 sweep (distinct scriptPubKeys).
2. m_{joint} is NOT confined to a tiny envelope ($\sim 2.9e76$ range).

The 5D joint lattice therefore cannot succeed as written.

However, the verification effort produced the strongest lead in the entire LOG2 research thread: the P135 sweep is a 21-input, mixed-signer batch with 10 KNOWN (d, k) pairs from one signing session. The exact RFC6979 matches at puzzles 85/90/120/125 prove that at least one of the signing tools used was deterministic. Classifying the signing scheme per input, then predicting k_{135} , is a concrete, testable next step — and it does not depend on any lattice precision numerology.

4. Data

```
txid    = 17e4e323cfbc68d7f0071cad09364e8193eedf8fefbcbdb8a21b4b65717a4b3d3
P135    = input 14, address 16RGFo6hj9ym6Pj7N5H7L1NR1rVPJyw2v
inputs  = puzzles 65,70,75,...,160 (every 5th), all SIGHASH_ALL (0x01)
solved  = 65,70,75,80,85,90,95,100,105,110,115,120,125,130 (keys known)
RFC6979 matches = {85, 90, 120, 125}

r_135   = c86bec9faea4892fd98d718bdfc770d0d11c3d6bfd4328f25fe9b06bfadb9650
s_135   = 224a322e81c044d341521f65fabdfa86d84673fb55ed7533862e37f7724931fa
z_135   = 92886faaf53f90a5c03d6af773a726e75097179306b980e5d28772e612e00fc7
```

```
N      = 115792089237316195423570985008687907852837564279074904382605163141518161494337
d_135  = (k_135 * s_135 - z_135) * inverse(r_135) mod N    [candidate check]

m_joint bounds (verified):
  min = -13948889879783289e60 ~ -1.39e76
  max =  1528799191624686e61 ~  1.53e76
(full precision in LOG2_organized.pdf section 9.2)
```